

Relatório de Integração — MOM e Arquitetura Event-Driven

Laboratório 5 | PUC Minas | Pedro Barbosa | Maio 2026

ESCOLHA DA FERRAMENTA MOM

Adotamos RabbitMQ como Message-Oriented Middleware (MOM) do projeto. A escolha foi motivada por três fatores principais. Primeiro, maturidade e simplicidade operacional: o RabbitMQ oferece suporte nativo a múltiplos padrões de troca (direct, topic, fanout, headers) com configuração mínima, adequado ao escopo acadêmico do laboratório. Segundo, suporte de primeira classe no ecossistema NestJS por meio da biblioteca `@golevelup/nestjs-rabbitmq`, que provê o decorator `@RabbitSubscribe` e gerencia automaticamente a criação de filas e bindings. Terceiro, o modelo de topic exchange atende exatamente ao padrão Publish/Subscribe com roteamento seletivo por routing key, evitando a necessidade de um filtro de mensagens no consumidor.

PADRÃO UTILIZADO

O padrão adotado é Publish/Subscribe com roteamento por tópico. Um único exchange durable do tipo topic (domain_events) concentra todos os eventos de domínio. Cada evento carrega uma routing key idêntica ao seu nome (ex.: group_purchase.created). Consumidores declaram filas duráveis e bindings específicos, recebendo apenas os eventos de seu interesse sem que o produtor precise conhecê-los. Esse desacoplamento permite adicionar novos consumidores sem alterar nenhum código existente. Dentro do domínio, seguimos o padrão de Outbox implícito: o agregado GroupPurchase acumula eventos internamente, o use case persiste o estado no banco e só então publica os eventos, evitando mensagens órfãs em caso de falha na persistência.

DECISÕES DE DESIGN

Decisão	Justificativa
Exchange único tipo topic	Centraliza todos os eventos num ponto de controle; novos módulos assinam com binding sem alterar produtores.
Filas duráveis por consumidor	Garante entrega mesmo que o consumidor esteja temporariamente offline (ex.: restart da aplicação durante um deploy).
Eventos gerados no agregado DDD	GroupPurchase_confirm() gera GroupPurchaseConfirmedEvent internamente, preservando invariantes de domínio e evitando lógica de negócio nos use cases.
Fire-and-forget com waitFor em testes	O produtor não bloqueia aguardando ACK do consumidor. No teste de integração, um helper waitFor() faz polling local no mock para verificar que o consumidor processou a mensagem.

DESAFIOS ENCONTRADOS

- **Corrida entre persistência e publicação:** Se a publicação ocorresse antes do commit no banco, um consumidor poderia consultar dados ainda não persistidos. Solução: publicação sempre após o retorno do `repositório.save()`.
- **Testes de integração com broker real:** Testes unitários com mock do `AmqpConnection` não validavam o roteamento real do exchange. Criamos um módulo de teste separado (`jest.integration.config.js`) que exige RabbitMQ ativo via docker-compose, validando o fluxo end-to-end.
- **Ordenação de eventos no agregado:** Quando o participante que entra completa o mínimo, o agregado gera `ParticipantJoinedEvent` e `GroupPurchaseConfirmedEvent` na mesma chamada. Ambos são publicados via `publishAll()`, respeitando a ordem de geração.
- **Conexão assíncrona na inicialização:** O `RabbitMQModule` usa `connectionInitOptions: { wait: false }` em produção para não bloquear o boot da aplicação caso o broker demore a subir, enquanto nos testes de integração `wait: true` garante que os consumers estejam registrados antes do primeiro publish.

Broker: RabbitMQ 3 (topic exchange domain_events) | Lib: @golevelup/nestjs-rabbitmq v4.1.0 | Padrão: Publish/Subscribe com routing key por nome de evento